第6章 利用数组处理批量数据

Part A

C语言的数据类型 （P43 图3.4）

- 数据类型
  - 基本类型
    - 整型
      - 基本整型 int
      - 短整型 short
      - 长整形 long
      - 双长整型* long long
      - 字符型 char
      - 布尔型* _Bool
    - 浮点型
      - 单精度浮点型 float
      - 双精度浮点型 double、long double
      - 复数浮点型 _Complex _Imaginary
  - 枚举类型（§9）
  - 空类型
  - 派生类型
    - 指针类型（§8）
    - 数组类型（§6）
    - 结构体类型（§9）
    - 共用体类型（§9）
    - 函数类型（§7）

算术类型

纯量类型

组合类型

有*的是C99所增加的。

zxl@xmu

为何需要使用数组

- 7个裁判打分，排序显示，取平均值作为选手的得分。
  - **实现：定义7个变量，分别存储各裁判的给分。**
- 500个观众给分，排序显示，取平均值作为选手的得分。
  - ***实现：定义500个变量？？***
- 又如：长整数各数位逆序

【存在问题】

1. 繁琐;

2. 无法反映出这些数据间的内在联系

为何需要使用数组

- 7个裁判打分，排序显示，取平均值作为选手的得分。
  - 实现：定义7个变量，分别存储各裁判的给分。
- 500个观众给分，排序显示，取平均值作为选手的得分。
  - *实现：定义500个变量？？*

【解决办法】

1. 用同一个名字（即数组名）来代表这些同一性质的数据（即数组），如s；

2. 在名字的右下角加一个数字（即下标）来表示这是第几个观众给的分，如$s_1,s_2,s_3,\ldots,s_{500}$，在C中写作：s[1],s[2],s[3],…,s[500]

数组

- 数组 —— 一组有序数据的集合。数组中各数据的排列是有一定规律的，下标代表数据在数组中的序号。
  - 数组中的元素都属于同一种数据类型。
  - 用一个数组名和下标来唯一地确定数组中的元素。
  - 数组可以是一维的或多维的。

a数组

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

目录

6.1 怎样定义和引用一维数组

6.2 怎样定义和引用二维数组

6.3 字符数组

6.1 怎样定义和引用一维数组

怎样定义一维数组

- 一维数组是数组中最简单的，它的元素只需要用数组名加一个下标，就能惟一确定。
- 要使用数组，必须在程序中先定义数组。

【定义方式】

类型符　数组名 [整型表达式]；

例如：int a[10];

a数组

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

【说明】

- 类型符：数组中每个元素的数据类型。
- 数组名：即标识符，要求符合标识符命名规则。
- 整型表达式：元素的个数，即数组长度。
  - 下标是从0开始的；

数组长度

类型符　数组名 [整型表达式] ;

- C89及之前，数组长度只能是常量表达式，即常量表达式中可包含常量和符号常量，但不能包含变量（C90改为数组长度在编译时要已知）。例如：**int a[4+6];**
- C99支持变长数组（编译期可变，限auto型，可使用const，不能初始化），即允许用变量对数组的大小进行定义，但程序运行过程中数组的大小不能会再改变（数组长度在运行时已知，但不支持运行期可变的**动态数组**）。

  例如：**int　n=4;　int　a[n];　　//不能是全局变量**
  - C99允许在被调用的函数（不包含主函数）中用形参变量或其表达式定义数组的长度，但被定义数组不能为静态（static）存储方式。例如：

```
void func(int n)
{ int a[2*n];
  ......
}
```

怎样引用一维数组

- 只能引用数组元素而不能一次整体调用整个数组全部元素的值

【数组元素引用形式】

数组名[*下标*]

例如：

```
int n=5, a[10];
a[1]=3*n;
a[0]=a[5]+a[7]-a[2*3];
a[n]=20; a[++n]=10;
```

a=0; ✗

- 数组下标*从0开始*，所以最大下标为 (数组长度-1)。
- 下标：可以是整型常量/整型表达式。
- 注意：下标值应当在已定义的范围内。*C不做下标越界检查！*越界访问有可能导致系统崩溃或异常的运行结果！

【例6.1】对10个数组元素依次赋值为0,1,2,3,4,5,6,7,8,9，要求按逆序输出。

【解题思路】

- 定义一个长度为10的数组，元素均为整型
- 要赋的值是从0到9，可以用循环来赋值
- 用循环按下标从大到小输出这10个元素

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
0	1	2	3	4	5	6	7	8	9

```
#include <stdio.h>
int main()
 {  int i,a[10];
    for (i=0; i<=9;i++)
        a[i]=i;
    for(i=9;i>=0; i--)
        printf("%d ",a[i]);
    printf("\n");
    return 0;
 }
```

使a[0]～a[9]的值为0～9

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
0	1	2	3	4	5	6	7	8	9

```
#include <stdio.h>
int main()
 {  int i,a[10];
    for (i=0; i<=9;i++)
        a[i]=i;
    for(i=9;i>=0; i--)
        printf("%d ",a[i]);
    printf("\n");
    return 0;
 }
```

```
9 8 7 6 5 4 3 2 1 0
```

先输出a[9]，最后输出a[0]

a[0]a[1]a[2]a[3]a[4]a[5]a[6]a[7]a[8]a[9]

0	1	2	3	4	5	6	7	8	9

一维数组的初始化

- 未初始化的数组元素初始值不可预料?!
  - 不同编译系统可能有所不同！有的会自动初始化为0，有的不会

数组的初始化：在定义数组的同时，给各数组元素赋值。

【“初始化列表”方法】：

类型说明符 数组名[常量表达式]= {常量1,…,常量n};

如：int a[10]={0,1,2,3,4,5,6,7,8,9};

- 按先后顺序依次对应赋值；
- 可以只给数组中的一部分元素赋初始值，*其它未给出的元素初始值为0*；

如： int a[10]={0,1,2,3,4}; 相当于int a[10]={0,1,2,3,4,0,0,0,0,0};

int a[10]={0}; 相当于int a[10]={0,0,0,0,0,0,0,0,0,0};

一维数组的初始化（2）

- 在对全部数组元素赋初始值时，可以不指定数组长度；

  例如：**int a[5]={0,1,2,3,4};**

  可以写成 **int a[ ]={0,1,2,3,4};**

- 如果被定义数组长度与提供的初值个数不同，则不能省略数组长度。

  错例： **int a[];**

  int a[5]={0,1,2,3,4,5};

【例6.2】用数组处理求Fibonacci数列问题。

【解题思路】

- 例5.8中用简单变量处理的，缺点不能在内存中保存这些数。假如想直接输出数列中第25个数，是很困难的。
- 如果用数组处理，每一个数组元素代表数列中的一个数，依次求出各数并存放在相应的数组元素中

```
#include <stdio.h>
int main()
 {  int i;
    int f[20]={1,1};
    for(i=2; i<20; i++)
       f[i]=f[i-2]+f[i-1];
    for(i=0; i<20; i++)
    {  if(i%5==0)
            printf("\n");
       printf("%12d", f[i]);
    }
   printf("\n");
```

```
           1           1           2           3           5
           8          13          21          34          55
          89         144         233         377         610
         987        1597        2584        4181        6765
```

【例6.3】有10个地区的面积，要求对它们按由小到大的顺序排列。

【解题思路】

- 排序的规律有两种：一种是“升序”，从小到大；另一种是“降序”，从大到小
- 把题目抽象为：“对n个数按升序排序”
- 采用起泡法排序

起泡排序思想

1. 将整个待排序序列划分成有序区和无序区，初始无序区包括所有n个待排序记录，有序区为空，位于无序区之后。
2. 一趟起泡排序：从前向后依次比较无序区中相邻两记录的关键字，若逆序则交换相邻记录，从而使得无序区中关键字最大的记录到了最后，这个记录划入有序区，无序区相应减少1个记录。
3. 如果一趟起泡排序中没有进行过交换记录的操作，则算法结束，否则重复执行步骤2至多n-1趟，直到无序区仅剩1个记录为止。

```
for(i=0;i<5;i++)
    if (a[i]>a[i+1]) {
        t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

	第1次比较	第2次比较	第3次比较	第4次比较	第5次比较
a[0]	3	3	3	3	3
a[1]	8	8	2	2	2
a[2]	2	2	8	5	5
a[3]	5	5	5	8	4
a[4]	4	4	4	4	8
a[5]	7	7	7	7	7

无序区

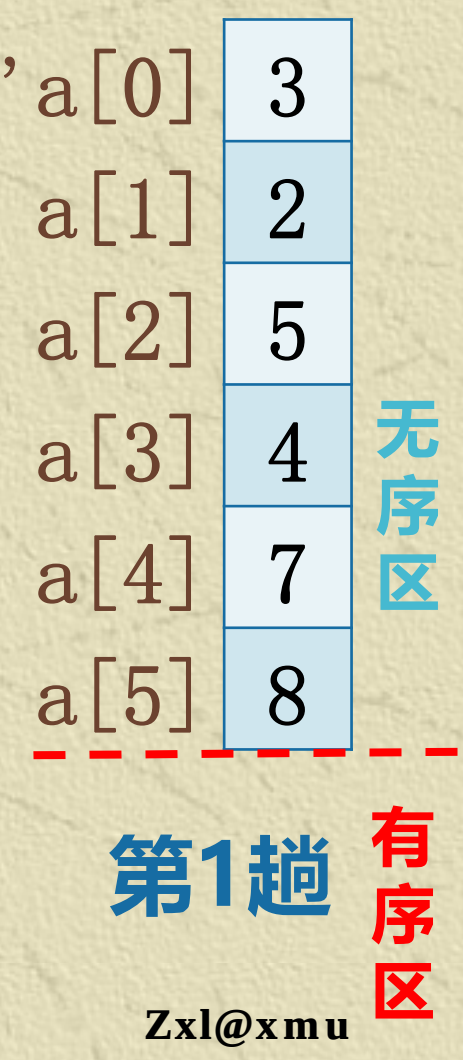
有序区

第1趟起泡排序

大数沉淀，小数起泡

起泡排序思想

1. 将整个待排序序列划分成有序区和无序区，初始无序区包括所有n个待排序记录，有序区为空，位于无序区之后。
2. 一趟起泡排序：从前向后依次比较无序区中相邻两记录的关键字，若逆序则交换相邻记录，从而使得无序区中关键字最大的记录到了最后，这个记录划入有序区，无序区相应减少1个记录。
3. 如果一趟起泡排序中没有进行过交换记录的操作，则算法结束，否则重复执行步骤2至多n-1趟，直到无序区仅剩1个记录为止。



```
for(i=0;i<4;i++)
    if (a[i]>a[i+1])  {
        t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

第1次比较　第2次比较　第3次比较　第4次比较

第2趟起泡排序

```
for(i=0;i<3;i++)
    if (a[i]>a[i+1])  {
        t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

	第1次比较	第2次比较	第3次比较
a[0]	2	2	2
a[1]	3	3	3
a[2]	4	4	4
a[3]	5	5	5
a[4]	7	7	7
a[5]	8	8	8

无序区

有序区

第3趟起泡排序

```
for(i=0;i<2;i++)
    if (a[i]>a[i+1]) {
      t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	2	a[0]	2
a[1]	3	a[1]	3
a[2]	4	a[2]	4
a[3]	5	a[3]	5
a[4]	7	a[4]	7
a[5]	8	a[5]	8

无序区

有序区

第1次比较　第2次比较

第4趟起泡排序

```
for(i=0;i<1;i++)
    if (a[i]>a[i+1]) {
        t=a[i];a[i]=a[i+1];a[i+1]=t; }
```

a[0]	2
a[1]	3
a[2]	4
a[3]	5
a[4]	7
a[5]	8

无序区

有序区

第1次比较

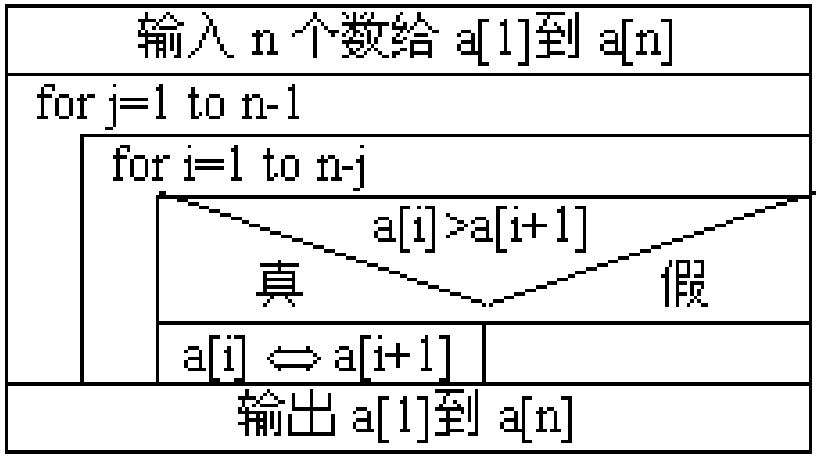
第5趟起泡排序

```
for(i=0;i<5;i++)
   if (a[i]>a[i+1])
      { ……}
```

```
for(i=0;i<4;i++)
   if (a[i]>a[i+1])
      { ……}
```

……

```
for(i=0;i<1;i++)
   if (a[i]>a[i+1])
      { ……}
```




```
for(j=0;j<5;j++)
   for(i=0;i<5-j;i++)
      if (a[i]>a[i+1])
         { ……}
```

```
int a[10];
int i,j,t;
printf("input 10 numbers :\n");
for (i=0;i<10;i++)
  scanf("%d",&a[i]);
printf("\n");
for(j=0; j<9; j++)
  for(i=0; i<9-j; i++)
    if (a[i]>a[i+1])
      {t=a[i];a[i]=a[i+1];a[i+1]=t;}
printf("the sorted numbers :\n");
for(i=0;i<10;i++)
  printf("%d ",a[i]);
printf("\n");
```

```
input 10 numbers :
34 67 90 43 124 87 65 99 132 26

the sorted numbers :
26 34 43 65 67 87 90 99 124 132
```

```
printf("\n");
for(j=0; j<9; j++)
  for(i=0; i<9-j; i++)
    if (a[i]>a[i+1])
      {t=a[i];a[i]=a[i+1];a[i+1]=t;}
printf("the sorted numbers :\n");
for(i=0;i<10;i++)
  printf("%d ",a[i]);
printf("\n");
```

6.2 怎样定义和引用二维数组

二维数组举例

	队员1	队员2	队员3	队员4	队员5	队员6
1分队	2456	1847	1243	1600	2346	2757
2分队	3045	2018	1725	2020	2458	1436
3分队	1427	1175	1046	1976	1477	2018

float pay[3][6];

- 二维数组常称为矩阵。

二维数组的定义

【定义形式】

类型说明符 数组名[常量表达式][常量表达式];

例如：**float a[3][4];** //定义了一个3行4列的数组。

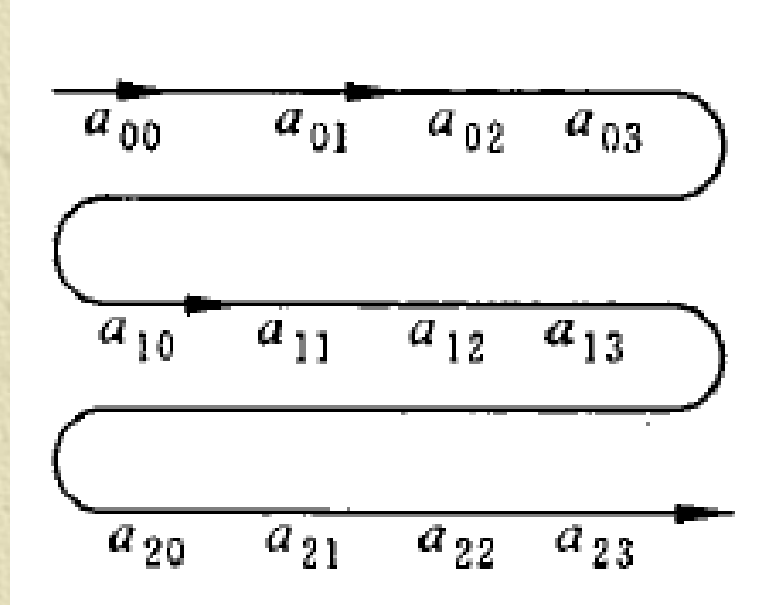
- C中，可将一个二维数组看成是一种特殊的一维数组，它的每个元素又是一个一维数组。

a	a[0] ——	a[0][0]	a[0][1]	a[0][2]	a[0][3]
	a[1] ——	a[1][0]	a[1][1]	a[1][2]	a[1][3]
	a[2] ——	a[2][0]	a[2][1]	a[2][2]	a[2][3]

  - **a[0],a[1],a[2]**可以看作一维数组名

二维数组的定义（2）

- 二维数组的存放顺序：按行存放。



多维数组的定义

- 多维数组元素在内存中的排列顺序是：第一维的下标变化最慢，最后一维的下标变化最快。

例如：**float a[3][5][4] ;**

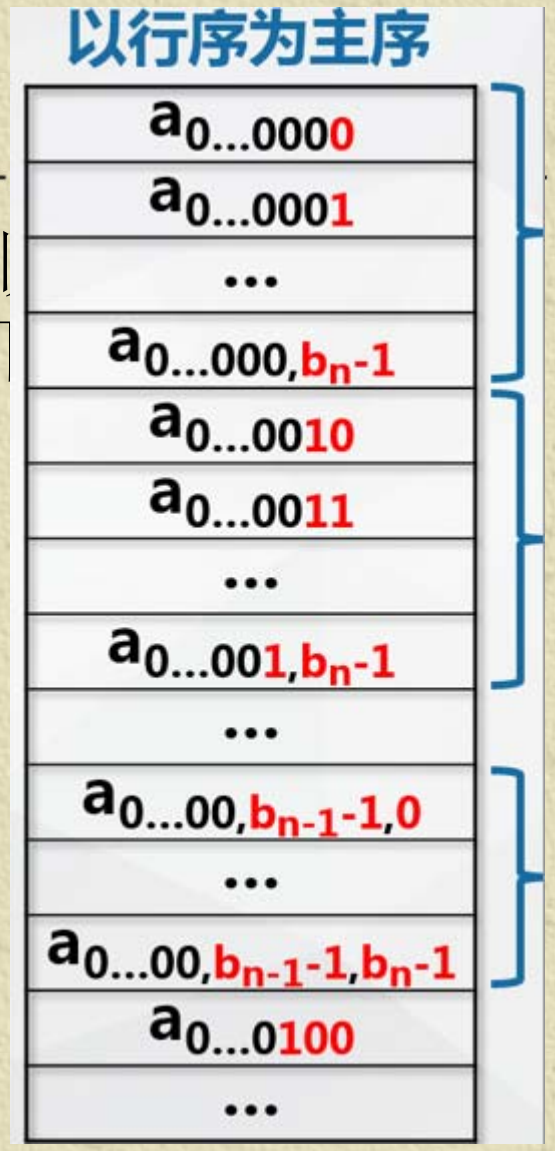
a[0][0][0]　a[0][0][1]　a[0] [0][2]　a[0] [0][3]
a[0][1][0]　a[0][1][1]　a[0][1][2]　a[0][1][3]
… …
a[2][4][0]　a[2][4][1]　a[2][4][2]　a[2][4][3]

多维数组的定义

- 多维数组元素在内存中的排列顺
的下标变化最慢，最后一维的下

例如：float a[3][5][4] ;

a[0][0][0] a[0][0][1] a[0] [0][2]
a[0][1][0] a[0][1][1] a[0][1][2]
… …
a[2][4][0] a[2][4][1] a[2][4][2]



二维数组的引用

【二维数组元素的表示形式】

数组名[下标][下标]

例如：

```
b[1][2]=a[2][3]/2;    合法
for(i=0;i<m;i++)
    printf("%d,%d\n",a[i][0],a[0][i]);合法
```

二维数组的初始化

- 可以分行赋初值，也可以不分行赋初值。

例如：**int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};**

等价于 **int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};**

- 正是由于多维数组的存储方式，才允许有这种初始化方法。但第一种较清晰。

- 可以只对部分元素赋初值，其余元素值自动为0；

例如： **int a[3][4]= {{1}, {5}, {9}};**

int a[3][4]={{1},{0,6},{0,0,11}};

int a[3][4]={{1},{5,6}};

int a[3][4]={{1},{},{9}};

二维数组的初始化（2）

- 若对全部元素都提供了初始数据，则定义数组时可以省略第一维的长度，但第二维的长度不可省。

  例如：**int a[][4]={0,0,0,0,1,1,1,1,2,2,2,2};**

- 对部分元素用分行赋初值时，也可以省略第一维的长度。

  例如：**int a[][4]={{0,0,3},{0},{0,10}};**

【例6.4】将一个二维数组行和列的元素互换，存到另一个二维数组中。

$$a=\begin{bmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{bmatrix} \Rightarrow b=\begin{bmatrix}1 & 4\\ 2 & 5\\ 3 & 6\end{bmatrix}$$

【解题思路】

- 可以定义两个数组：数组a为2行3列，存放指定的6个数；数组b为3行2列，开始时未赋值。
- 将a数组中的元素a[i][j]存放到b数组中的b[j][i]元素中
- 用嵌套的for循环完成

```
#include <stdio.h>
int main()
 {  int a[2][3]={{1,2,3},{4,5,6}};
    int b[3][2],i,j;
    printf("array a:\n");
    for (i=0; i<=1; i++)   处理a的一行中各元素
    { for (j=0; j<=2; j++) 处理a中某一列元素
      {  printf("%5d",a[i][j]); 输出a的各元素
         b[j][i]=a[i][j]; a元素值赋给b相应元素
       }
      printf("\n");
     }
```

```
    printf("array b:\n");
    for (i=0;i<=2;i++)
    {  for(j=0;j<=1;j++)
           printf("%5d",b[i][j]);
       printf("\n");
    }
    return 0;
}
```

输出b的各元素

```
array a:
    1    2    3
    4    5    6
array b:
    1    4
    2    5
    3    6
```

【例6.5】有一个3×4的矩阵，要求编程序求出其中值最大的那个元素的值，以及其所在的行号和列号。

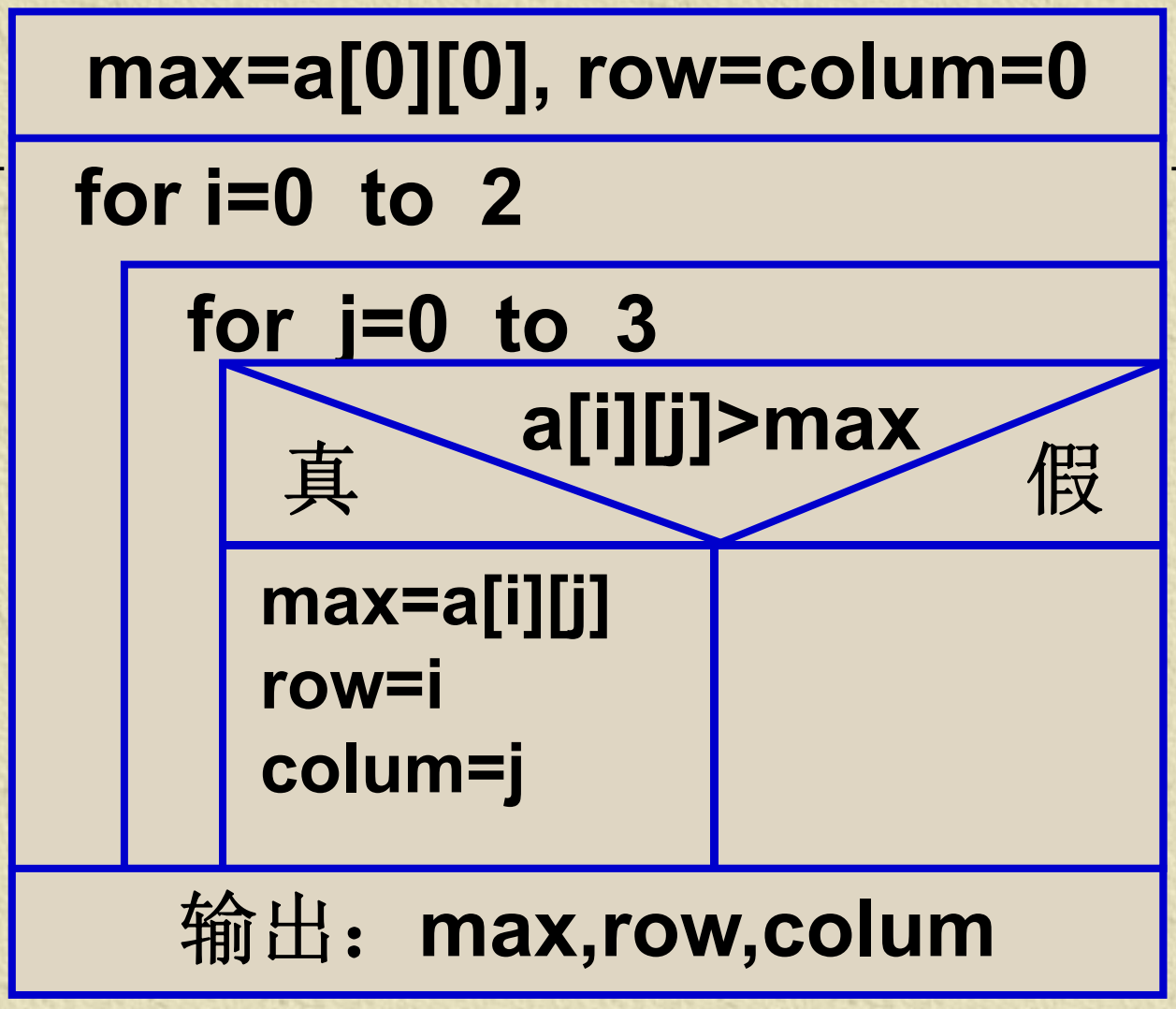
【解题思路】采用“打擂台算法”

- 先任意找出一个人站在台上，第2人上去与之比武，胜者留在台上
- 第3人与台上的人比武，胜者留台上，败者下台
- 以后，每一个人都是与当时留在台上的人比武，直到所有人都上台比过武为止，最后留在台上的是冠军

【例6.5】有一个3×4的矩阵，要求编程序求出其中值最大的那个元素的值，以及其所在的行号和列号。

【解题思路】采用“打擂台算法”

- max用来存放当前已知的最大值
- 先把a[0][0]的值赋给变量max;
- a[0][1]与max比较，如果a[0][1]>max，则表示a[0][1]是已经比过的数据中值最大的，把它的值赋给max，取代了max的原值
- 以后依此处理，最后max就是最大的值


max=a[0][0], row=colum=0
for i=0 to 2
for j=0 to 3
a[i][j]>max
真
假
max=a[i][j]
row=i
colum=j
输出：max,row,colum

```
#include <stdio.h>
main()
{int i,j,row=0,colum=0,max;
 int a[3][4]={{1,2,3,4},{9,8,7,6},
                        {-10,10,-5,2}};
 max=a[0][0];
 for (i=0;i<=2;i++)
  for (j=0;j<=3;j++)
    if (a[i][j]>max) {
       max=a[i][j]; row=i; colum=j; }
  printf("max=%d\nrow=%d\n
          colum=%d\n",max,row,colum);
  return 0;
}
```

```
#include <stdio.h>
main()
{int i,j,row=0,colum=0,max;
 int a[3][4]={{1,2,3,4},{9,8,7,6},
                        {-10,10,-5,2}};
 max=a[0][0];
 for (i=0;i<=2;i++)
  for (j=0;j<=3;j++)
    if (a[i][j]>max) {
      max=a[i][j]; row=i; colum=j; }
 printf("max=%d\nrow=%d\n
        colum=%d\n",max,row,colum);
 return 0;
}
```

```
max=10
row=2
colum=1
```

记最大值

记行号

记列号